

## Memory Architecture

### CORE:

- keep square within a 2:1 ratio
- rows are **word lines**
- columns are **bit lines**
- data in and out on columns

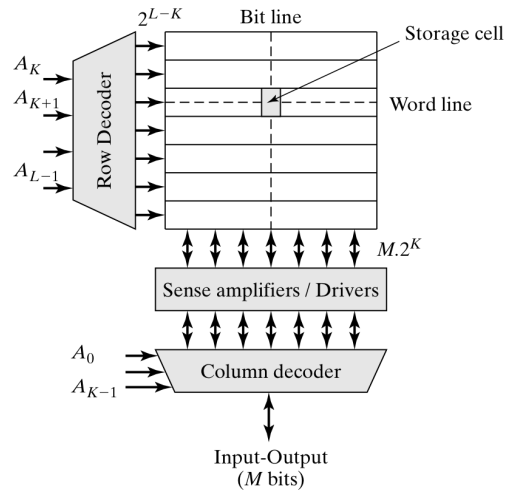
### DECODERS:

- needed to reduce total number of pins;  $N+M$  address lines for  $2^{N+M}$  bits of storage

Ex: if  $N+M=20 \rightarrow 2^{20} = 1\text{Mb}$

### MULTIPLEXING:

- used to select one or more columns for input or output of data

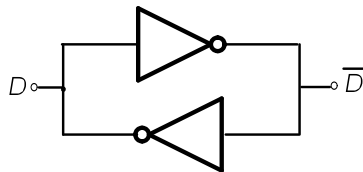


EECS141

Lecture #25

17

## Basic Static Memory Element



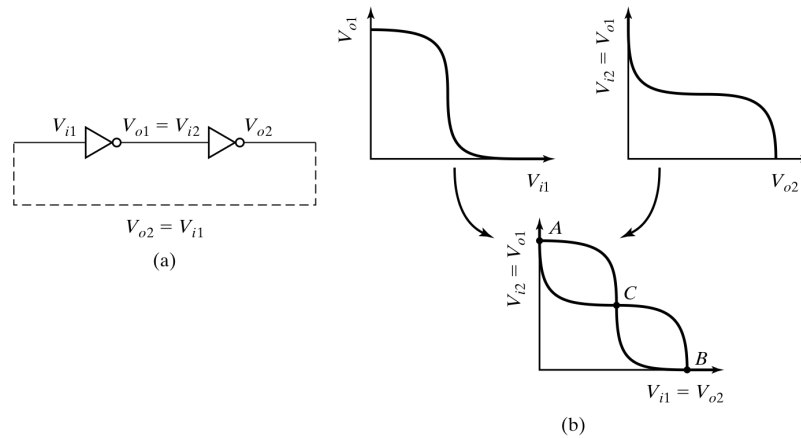
- If  $D$  is high,  $D_b$  will be driven low
  - Which makes  $D$  stay high
- Positive feedback

EECS141

Lecture #25

18

## Positive Feedback: Bi-Stability

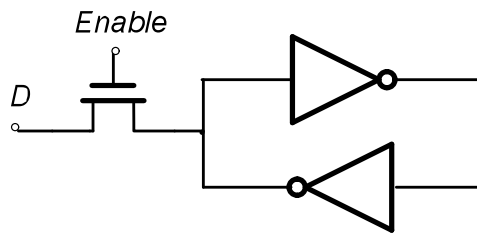


EECS141

Lecture #25

19

## Writing into a Cross-Coupled Pair



Access transistor must be able to overpower the feedback

EECS141

Lecture #25

20

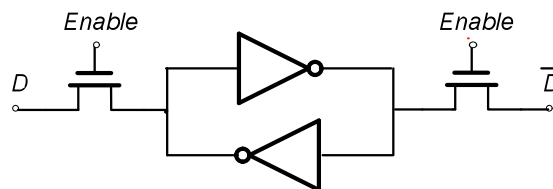
## Writing a "1"

EECS141

Lecture #25

21

## Memory Cell



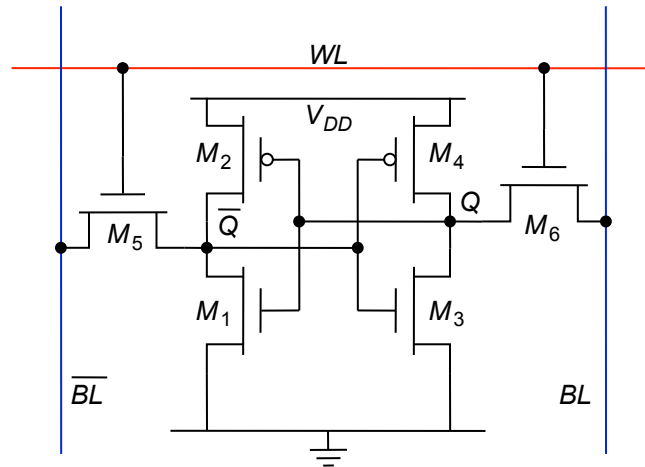
Complementary data values are written (read) from two sides

EECS141

Lecture #25

22

## 6-Transistor CMOS SRAM Cell



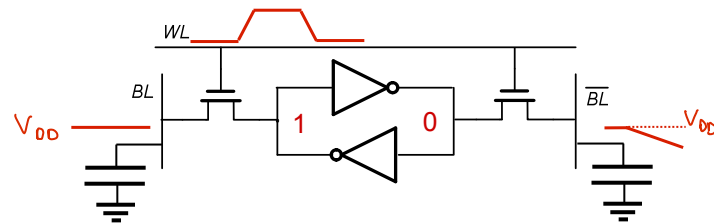
EECS141

Lecture #25

23

## SRAM Operation - Read

Read



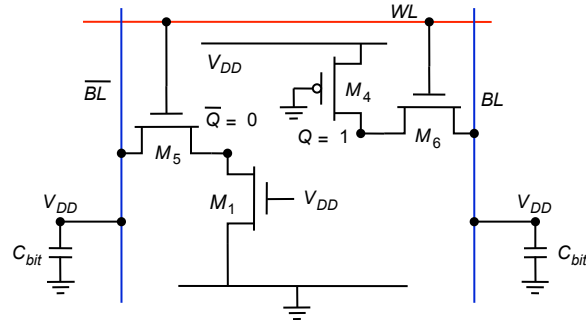
- Q\_b will get pulled up when WL first goes high
- Reading the cell should not destroy the stored value

EECS141

Lecture #25

24

## CMOS SRAM Analysis (Read)



$$k_{n,M5} \left( (V_{DD} - \Delta V - V_{Tn}) V_{DSATn} - \frac{V_{DSATn}^2}{2} \right) = k_{n,M1} \left( (V_{DD} - V_{Tn}) \Delta V - \frac{\Delta V^2}{2} \right)$$

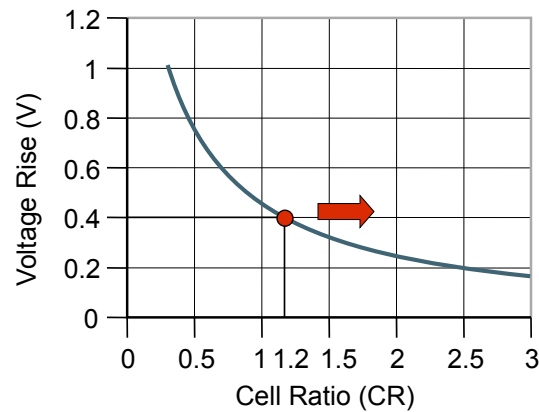
$$\Delta V = \frac{V_{DSATn} + CR(V_{DD} - V_{Tn}) - \sqrt{V_{DSATn}^2(1 + CR) + CR^2(V_{DD} - V_{Tn})^2}}{CR}$$

EE141

Lecture #25

25

## CMOS SRAM Analysis (Read)



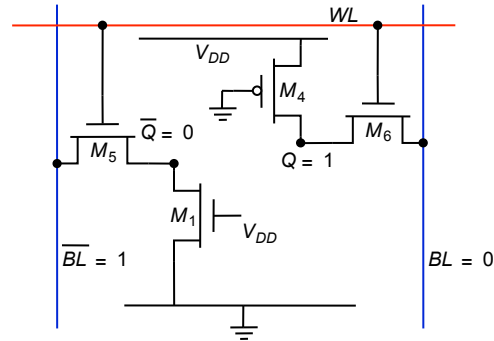
$$CR = \frac{W_1/L_1}{W_5/L_5}$$

EE141

Lecture #25

26

## CMOS SRAM Analysis (Write)



$$k_{n,M6} \left( (V_{DD} - V_{Tn}) V_Q - \frac{V_Q^2}{2} \right) = k_{p,M4} \left( (V_{DD} - |V_{Tp}|) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right)$$

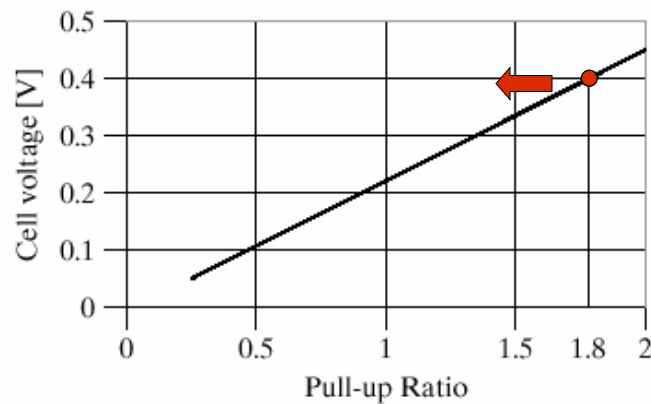
$$V_Q = V_{DD} - V_{Tn} - \sqrt{(V_{DD} - V_{Tn})^2 - 2 \frac{\mu_p}{\mu_n} PR \left( (V_{DD} - |V_{Tp}|) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right)}$$

EE141

Lecture #25

27

## CMOS SRAM Analysis (Write)

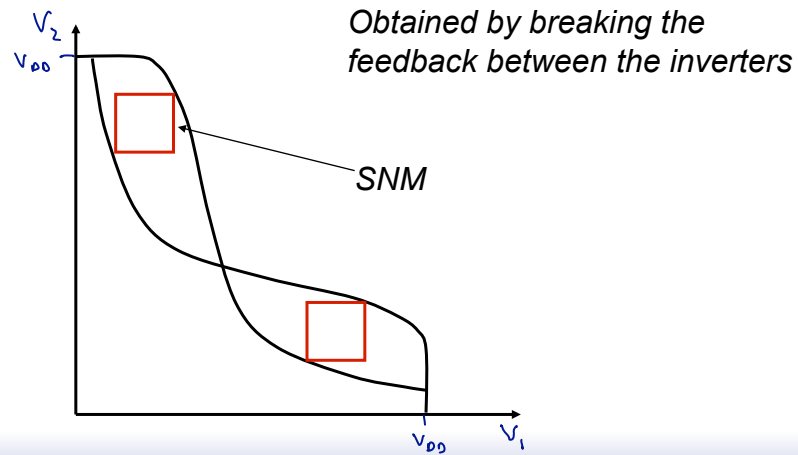


EE141

Lecture #25

28

## Read Static Noise Margin

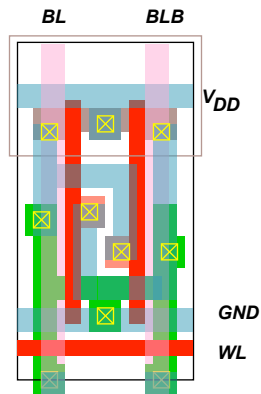


EECS141

Lecture #25

29

## 6T-SRAM — Layout



Compact cell  
 Bitlines: M2  
 Wordline: bootstrapped in M3

EECS141

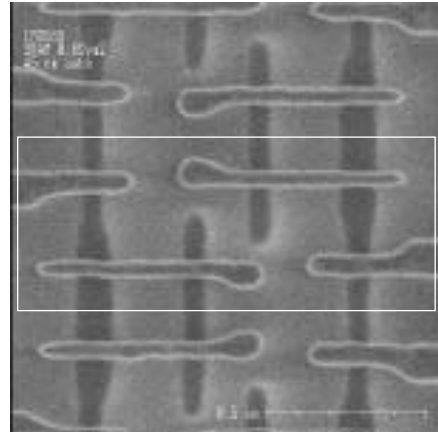
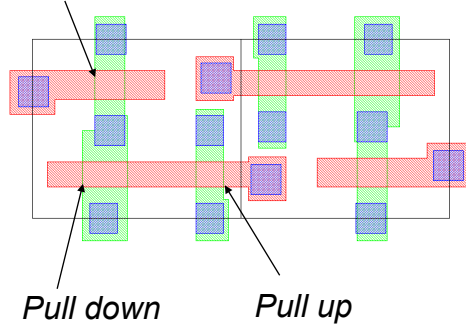
Lecture #25

30

## 65nm SRAM

□ ST/Philips/Motorola

*Access Transistor*

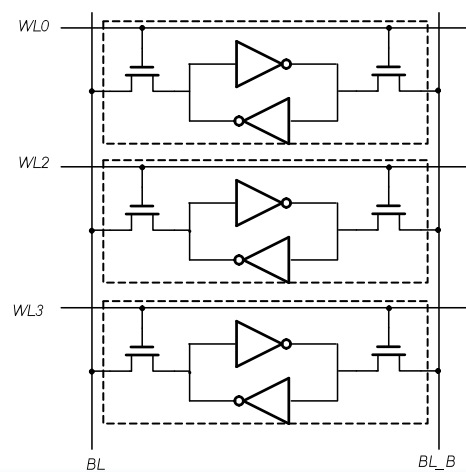


EECS141

Lecture #25

31

## SRAM Column



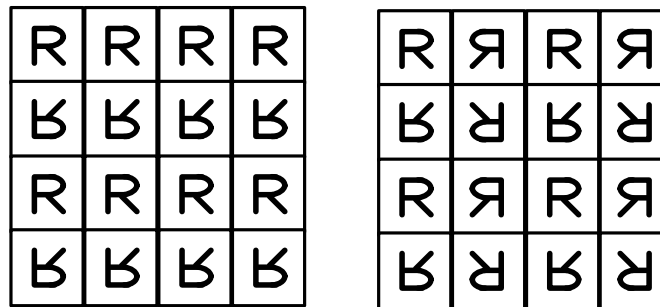
EECS141

Lecture #25

32



## SRAM Array Layout

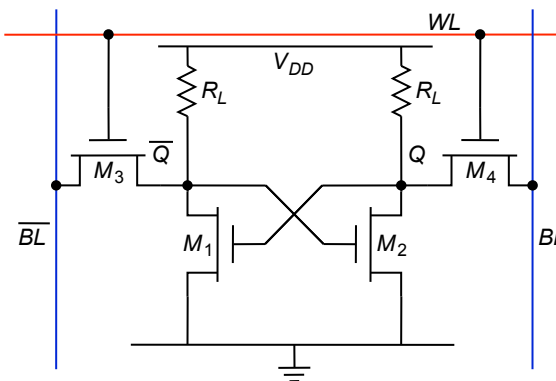


EECS141

Lecture #25

33

## Resistive Pull-Ups



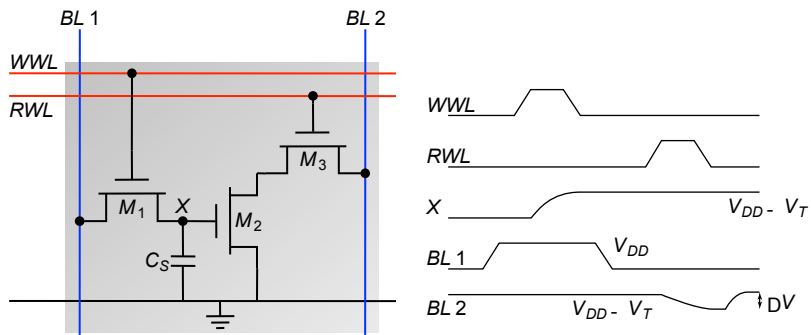
Static power dissipation -- Want  $R_L$  large  
 Bit lines precharged to  $V_{DD}$  to address  $t_p$  problem

EECS141

Lecture #25

34

## 3-Transistor DRAM Cell



No constraints on device ratios

Reads are non-destructive

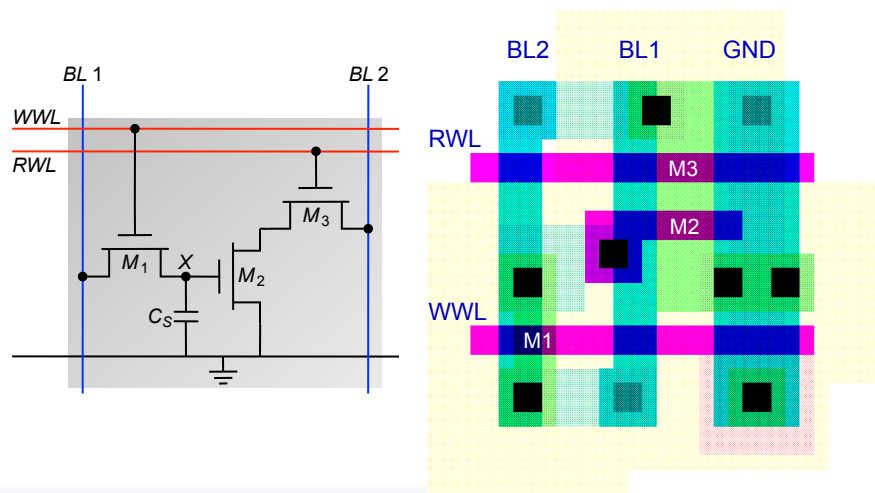
Value stored at node X when writing a "1" =  $V_{WWL} - V_{Tn}$

EE141

Lecture #25

35

## 3T DRAM Layout

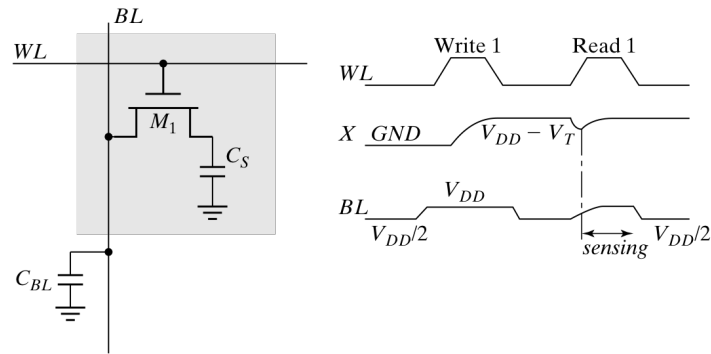


EE141

Lecture #25

36

## 1-Transistor DRAM Cell



Write:  $C_S$  is charged or discharged by asserting WL and BL.  
 Read: Charge redistribution takes places between bit line and storage capacitance

$$\Delta V = V_{BL} - V_{PRE} = V_{BIT} - V_{PRE} \frac{C_S}{C_S + C_{BL}}$$

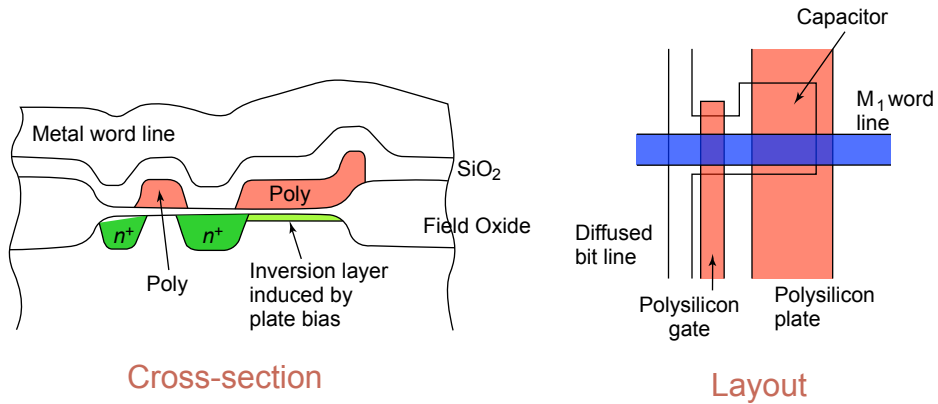
Voltage swing is small; typically around 250 mV.

EE141

Lecture #25

37

## 1T1C DRAM Cell



Cross-section

Layout

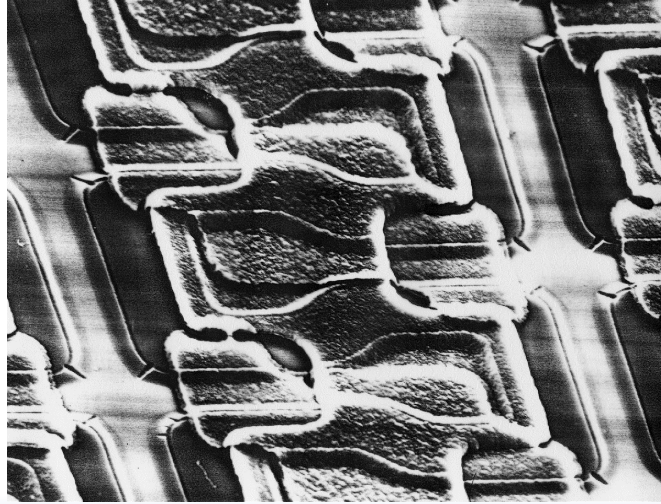
Uses Polysilicon-Diffusion Capacitance  
 Expensive in Area

EE141

Lecture #25

38

## Micrograph of 1T DRAM

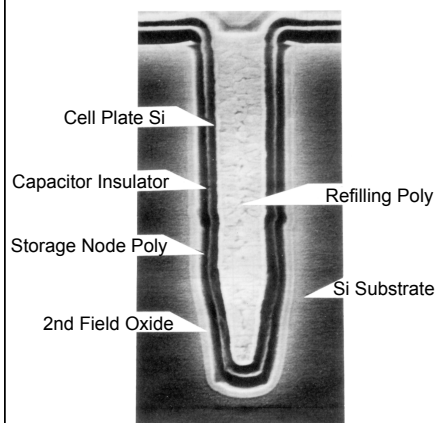


EECS141

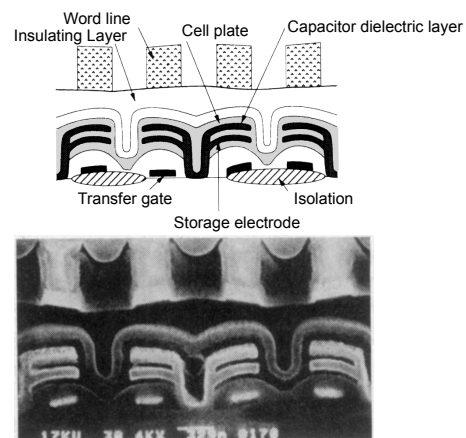
Lecture #25

39

## Advanced 1T DRAM Cells



Trench Cell



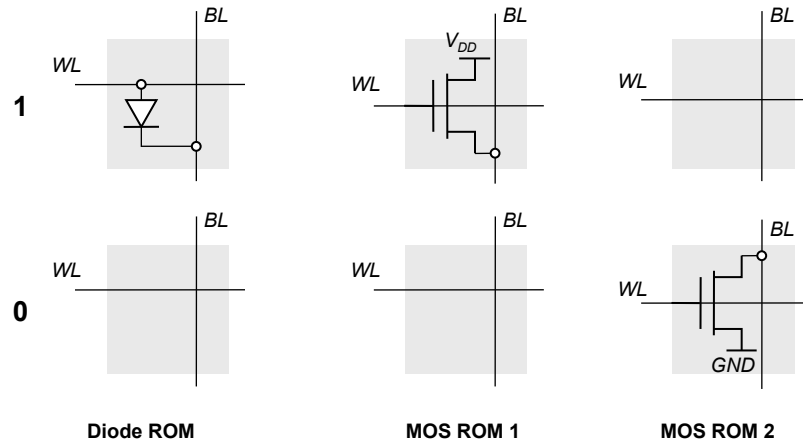
Stacked-capacitor Cell

EECS141

Lecture #25

40

## Read-Only Memory

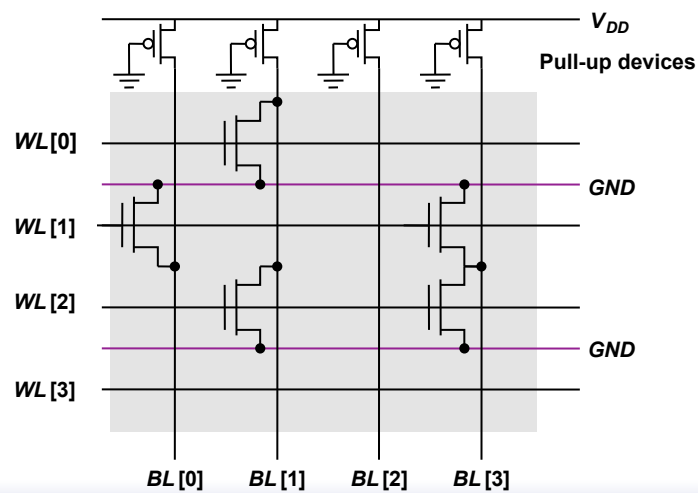


EECS141

Lecture #25

41

## MOS NOR ROM

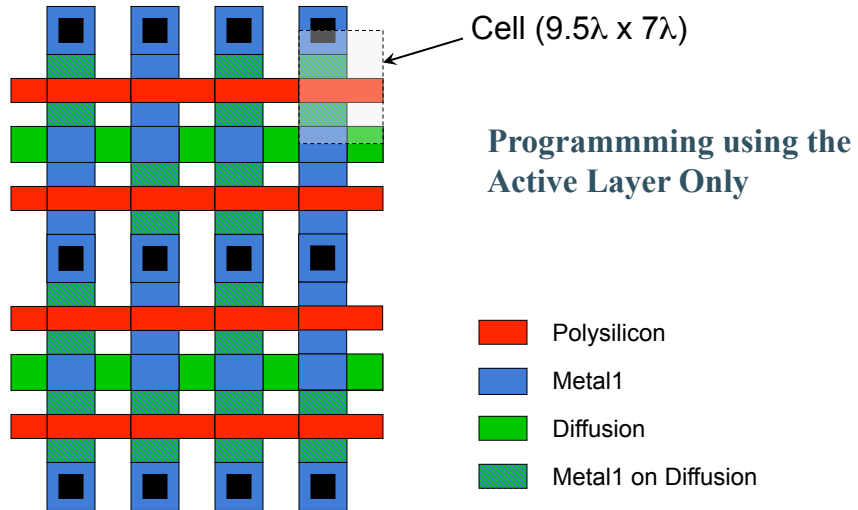


EECS141

Lecture #25

42

## MOS NOR ROM Layout

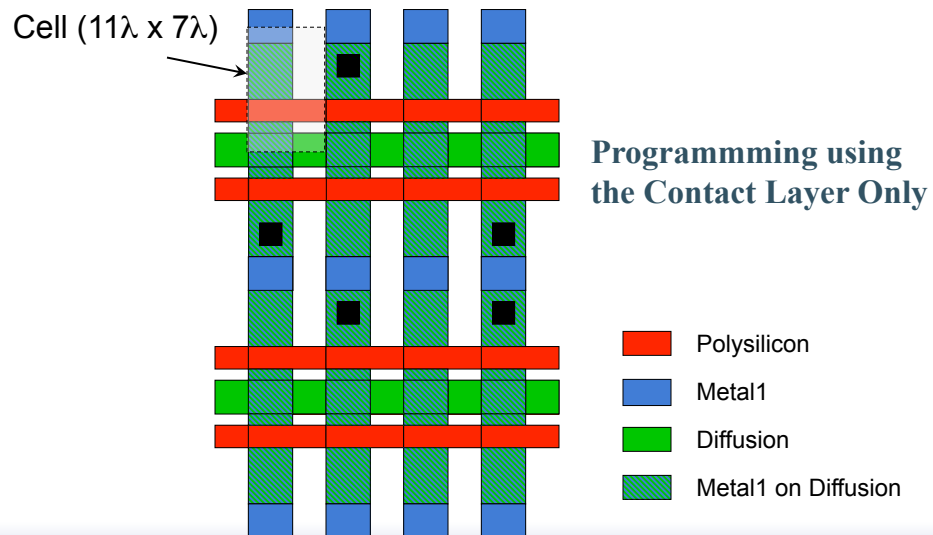


EECS141

Lecture #25

43

## MOS NOR ROM Layout

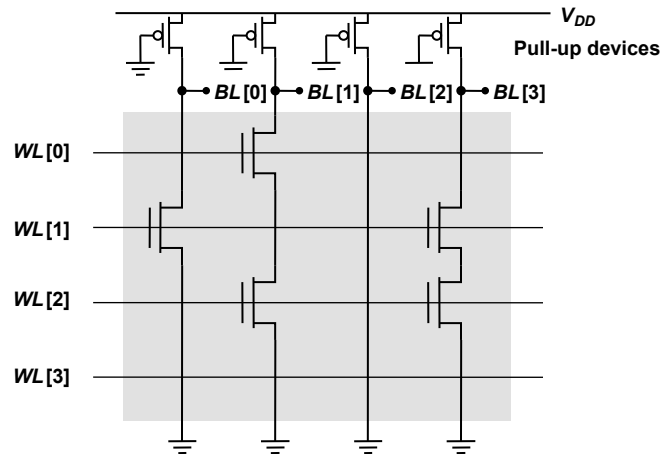


EECS141

Lecture #25

44<sup>44</sup>

## MOS NAND ROM



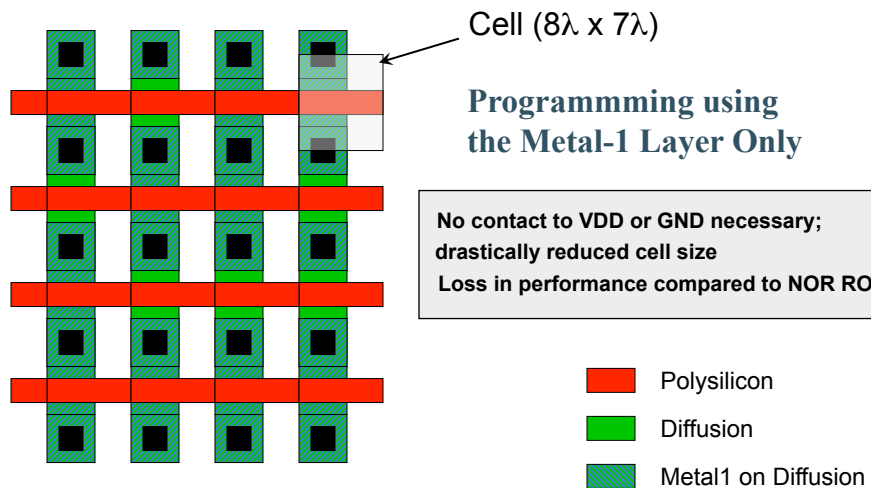
All word lines high by default with exception of selected row

EECS141

Lecture #25

45

## MOS NAND ROM Layout



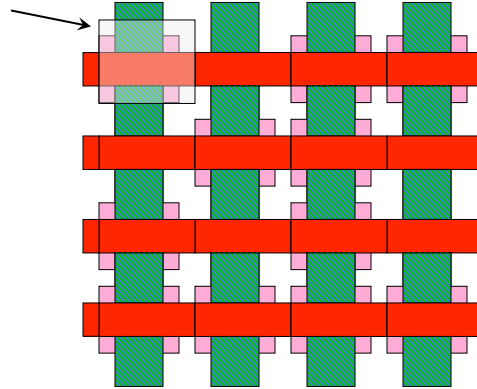
EECS141

Lecture #25

46

## NAND ROM Layout

Cell ( $5\lambda \times 6\lambda$ )



Programming using  
Implants Only

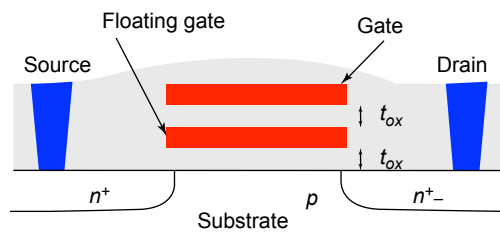
- Polysilicon
- Threshold-altering implant
- Metal1 on Diffusion

EECS141

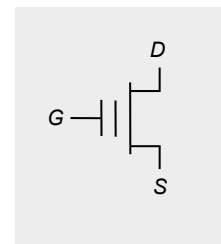
Lecture #25

47

## Non-Volatile Memories The Floating-gate transistor (FAMOS)



Device cross-section



Schematic symbol

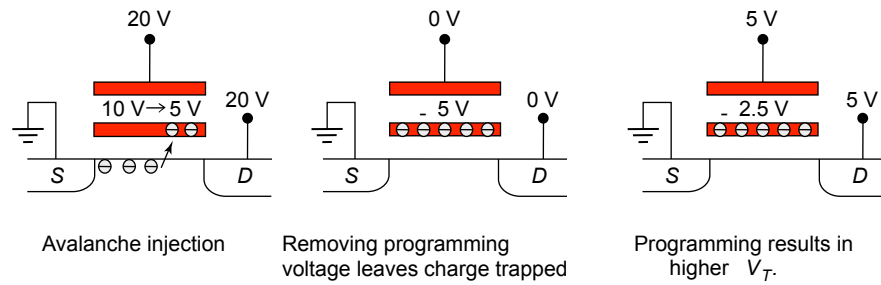
EECS141

Lecture #25

48



## Floating-Gate Transistor Programming

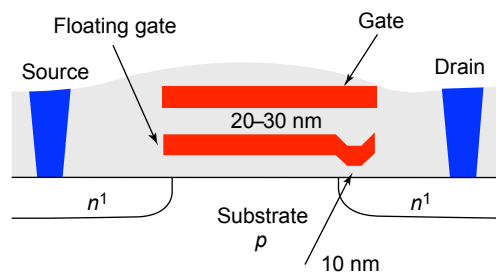


EECS141

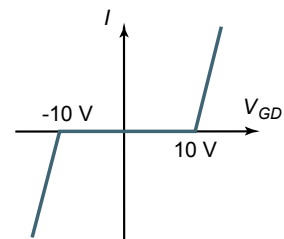
Lecture #25

49

## FLTOX EEPROM



FLTOX transistor

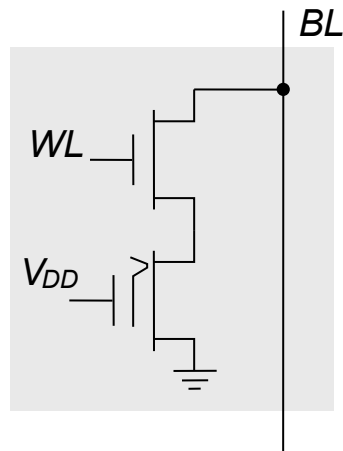
Fowler-Nordheim  
 $I$ - $V$  characteristic

EECS141

Lecture #25

50

## EEPROM Cell



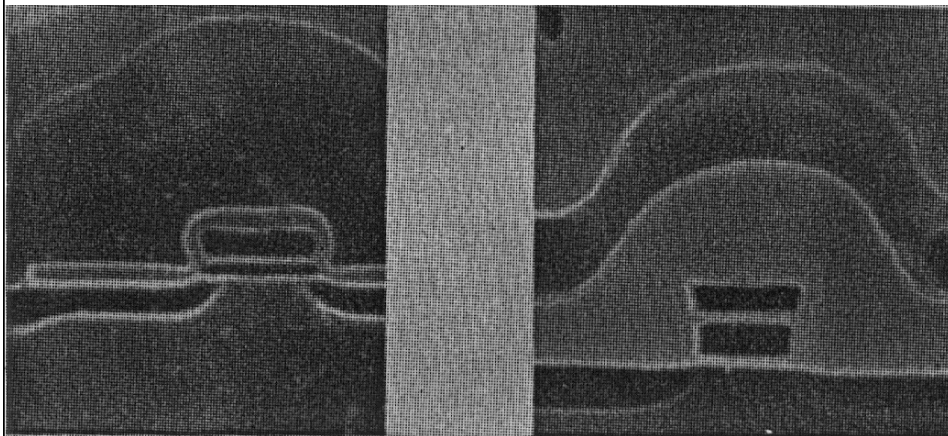
Absolute threshold control  
is hard  
Unprogrammed transistor  
might be depletion  
⇒ 2 transistor cell

EECS141

Lecture #25

51

## Cross Sections of NVM Cells



Flash

EPROM

*Courtesy Intel*

EECS141

Lecture #25

52

## Periphery

- ☐ Decoders
- ☐ Sense Amplifiers
- ☐ Input/Output Buffers
- ☐ Control / Timing Circuitry

EECS141

Lecture #25

53

## Row Decoders

Collection of  $2^M$  complex logic gates  
Organized in regular and dense fashion

(N)AND Decoder

$$WL_0 = A_0 A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8 A_9$$

$$WL_{511} = \bar{A}_0 \bar{A}_1 \bar{A}_2 \bar{A}_3 \bar{A}_4 \bar{A}_5 \bar{A}_6 \bar{A}_7 \bar{A}_8 \bar{A}_9$$

NOR Decoder

$$WL_0 = \overline{A_0 + A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7 + A_8 + A_9}$$

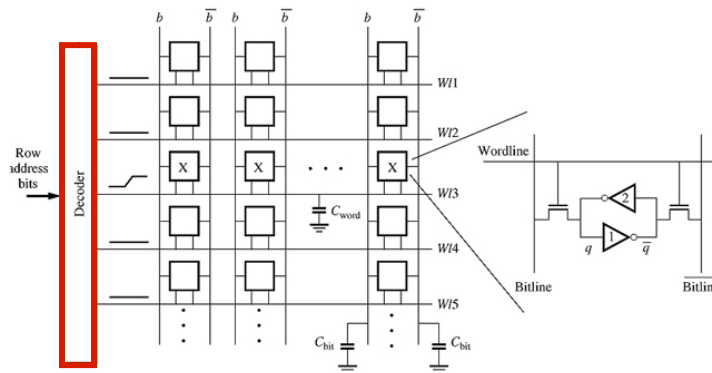
$$WL_{511} = \overline{A_0 + \bar{A}_1 + \bar{A}_2 + \bar{A}_3 + \bar{A}_4 + \bar{A}_5 + \bar{A}_6 + \bar{A}_7 + \bar{A}_8 + \bar{A}_9}$$

EECS141

Lecture #25

54

## Decoder Design Example



- Look at decoder for 256x256 memory block (8KBytes)

EECS141

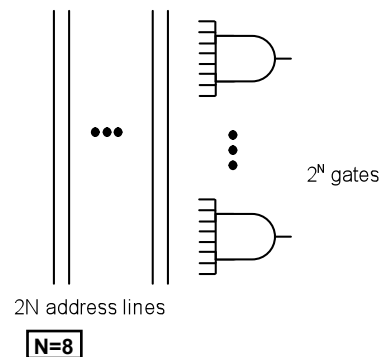
Lecture #25

55

## Problem Setup

- Goal: Build fastest possible decoder with static CMOS logic

- What we know
  - Basically need 256 AND gates, each one of them drives one word line



EECS141

Lecture #25

56

## Problem Setup (1)

- Each word line has 256 cells connected to it
- Total output load is  $256 \cdot C_{\text{cell}} + C_{\text{wire}}$
- Assume that decoder input capacitance is  $C_{\text{address}} = 4 \cdot C_{\text{cell}}$
- Each address drives  $2^8/2$  AND gates
  - A0 drives  $\frac{1}{2}$  of the gates, A0\_b the other  $\frac{1}{2}$  of the gates
- Neglecting  $C_{\text{wire}}$ , the fan-out on each one of the 16 address wires is:
 
$$BF = \frac{C_{\text{load}}}{C_{\text{in}}} = \frac{(2^8 / 2)(2^8 C_{\text{cell}})}{4C_{\text{cell}}} = 2^{13}$$

EECS141

Lecture #25

57

## Decoder Fan-Out

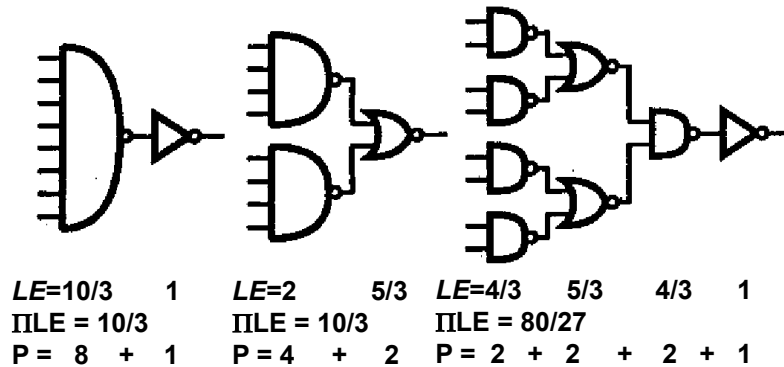
- FB of at least  $2^{13}$  means that we will want to use more than  $\log_4(2^{13}) = 6.5$  stages to implement the AND8
- Need many stages anyways
  - So what is the best way to implement the AND gate?
  - Will see next that it's the one with the most stages and least complicated gates

EECS141

Lecture #25

58

## 8-Input AND

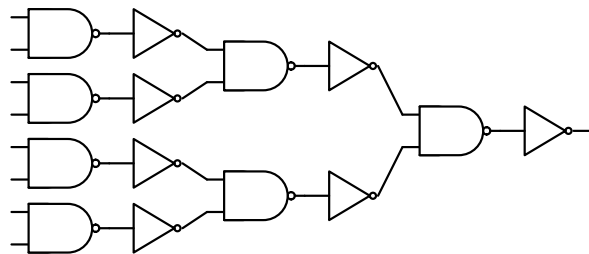


EECS141

Lecture #25

59

## 8-Input AND



- Using 2-input NAND gates
  - 8-input gate takes 6 stages
- Total LE is  $(4/3)^3 \approx 2.4$
- So PE is  $2.4 \cdot 2^{13}$  – optimal N of  $\sim 7.1$

EECS141

Lecture #25

60

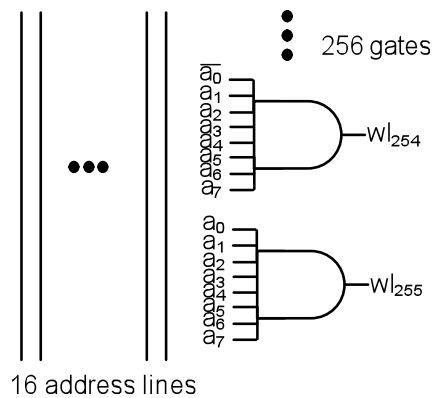
## Decoder So Far

### 256 8-input AND gates

- Each built out of tree of NAND gates and inverters

### Issue:

- Every address line has to drive 128 gates (and wire) right away
- Can't build gates small enough - Forces us to add buffers just to drive address inputs

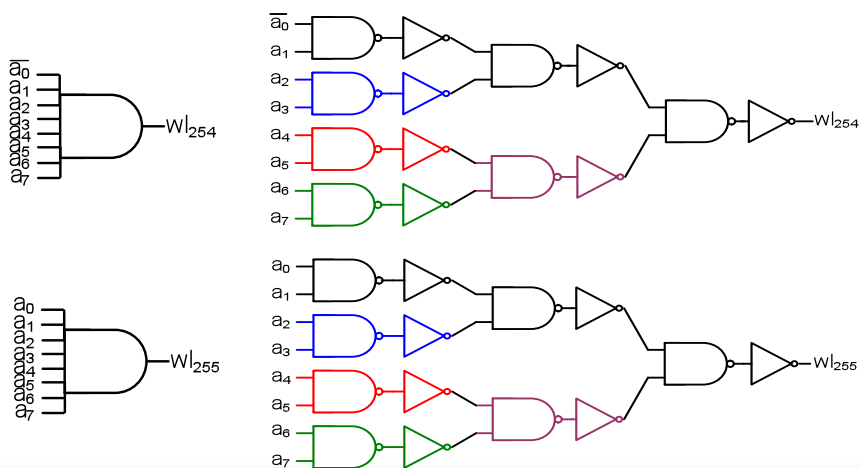


EECS141

Lecture #25

61

## Look Inside Each AND8 Gate



EECS141

Lecture #25

62

## Predecoders

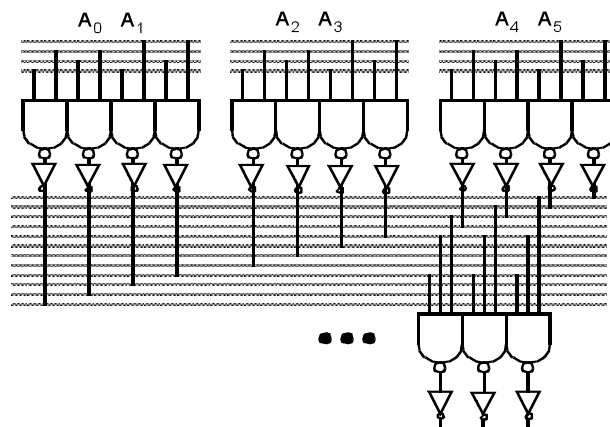
- Use a single gate for each of the shared terms
  - E.g., from  $A_0$ ,  $\overline{A_0}$ ,  $A_1$ , and  $\overline{A_1}$ , generate four signals:  $A_0A_1$ ,  $\overline{A_0}A_1$ ,  $A_0\overline{A_1}$ ,  $\overline{A_0}\overline{A_1}$
- In other words, we are decoding smaller groups of address bits first
  - And using the “predecoded” outputs to do the rest of the decoding

EECS141

Lecture #25

63

## Predecoder and Decoder



EECS141

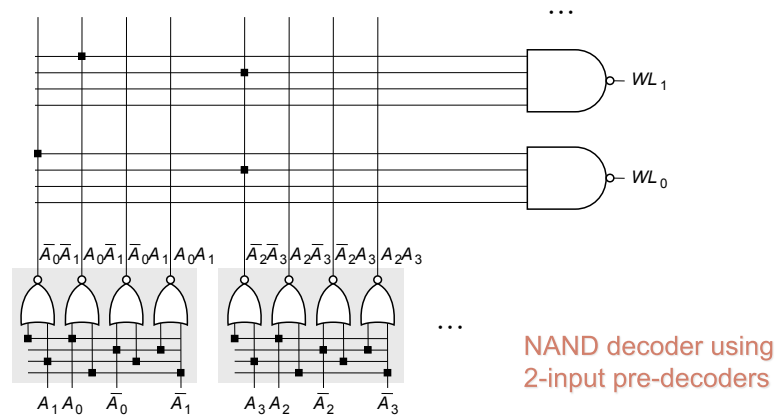
Lecture #25

64



## Hierarchical Decoders

*Multi-stage implementation improves performance*

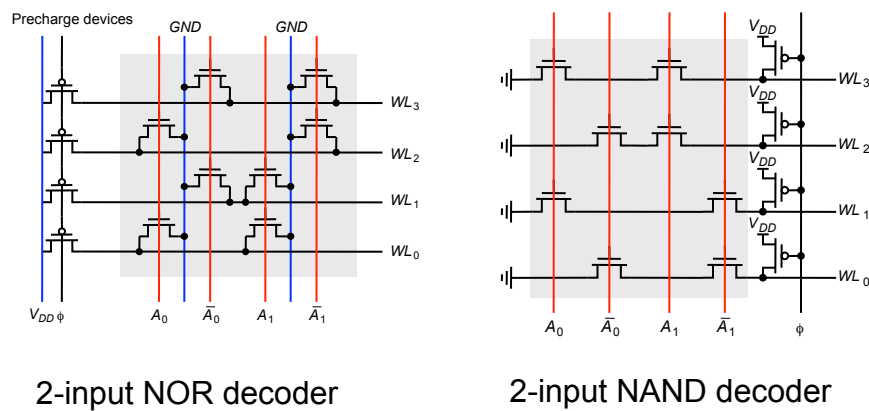


EECS141

Lecture #25

65

## Dynamic Decoders

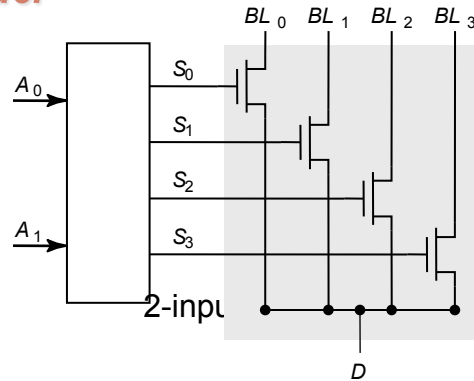


EECS141

Lecture #25

66

### 4-input pass-transistor based column decoder



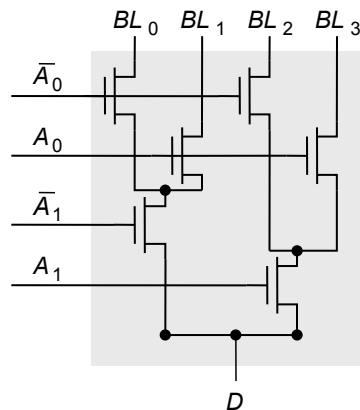
Advantages: speed ( $t_{pd}$  does not add to overall memory access time)  
 Only one extra transistor in signal path  
 Disadvantage: Large transistor count

EECS141

Lecture #25

67

### 4-to-1 tree based column decoder



Number of devices drastically reduced  
 Delay increases quadratically with # of sections; prohibitive for large decoders  
 Solutions: buffers  
 progressive sizing  
 combination of tree and pass transistor approaches

EECS141

Lecture #25

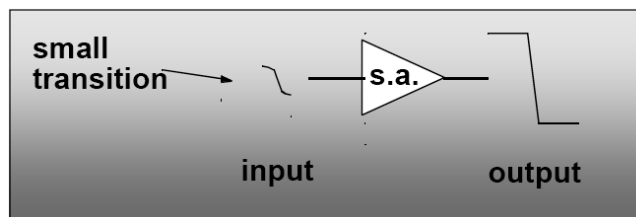
68

## Sense Amplifiers

$$t_p = \frac{C \cdot \Delta V}{I_{av}}$$

make  $\Delta V$  as small as possible  
 large (for  $C$ )  
 small (for  $I_{av}$ )

Idea: Use Sense Amplifier

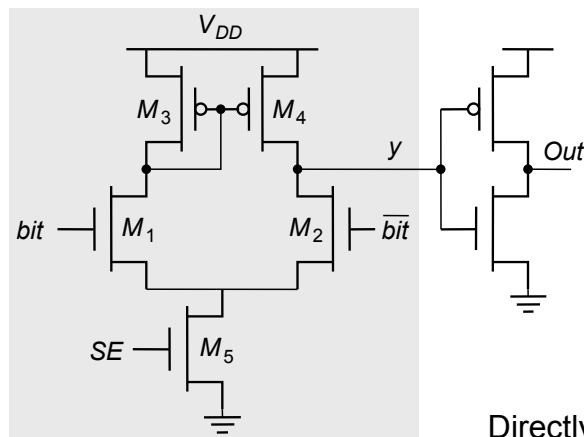


EECS141

Lecture #25

69

## Differential Sense Amplifier



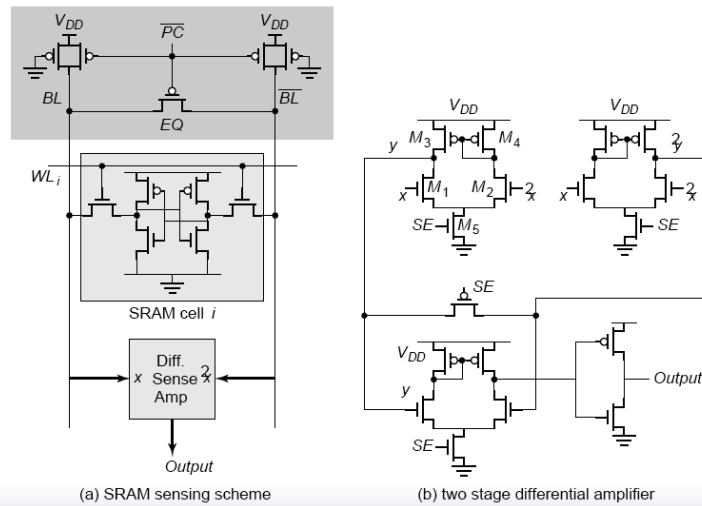
Directly applicable to  
SRAMs

EECS141

Lecture #25

70

## Differential Sensing – SRAM

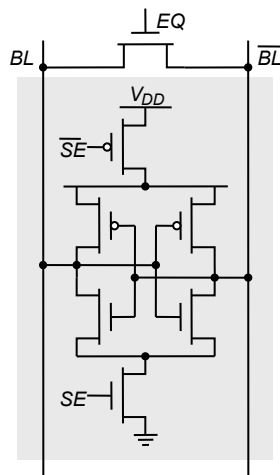


EECS141

Lecture #25

71

## Latch-Based Sense Amplifier (DRAM)



Initialized in its meta-stable point with EQ  
 Once adequate voltage gap created, sense amp enabled with SE  
 Positive feedback quickly forces output to a stable operating point.

EECS141

Lecture #25

72